

Rahul Sharma

Project Lead | Game Testing & Project Management

I lead game QA and project delivery end to end — scope, schedule, staffing and stakeholder sign-off — across PC, console, mobile and web, from test strategy through release. I run the Jira process and keep the daily numbers honest, even when that's inconvenient.

Delhi, India | +91-7217704787 | mrrahulsharma@yahoo.com | [linkedin.com/in/rahul-sharma-9a4661133](https://www.linkedin.com/in/rahul-sharma-9a4661133)

3000+

QA deliverables managed across PC, console, mobile and web

40+

game projects tested and released live

14,784 h

structured testing effort across 1,848 days

7

successful, revenue-generating international clients

10+

testers trained and mentored across all projects

Why this portfolio is different

- **Facts, not adjectives.** The headline numbers above reflect my delivery record across all engagements to date. The two deep-dive case studies in this portfolio go further: every figure in them traces to a specific source file — a Jira export, a daily log, a signed-off report — and where the source documents disagreed, I reconciled them and show the reconciled number.
- **Decisions, not just execution.** On Project B, I called the build *not ready for release* from the data (15 of 24 Highest/High defects unresolved) and set out a P0/P1 fix plan.
- **Ownership of mistakes.** On Project A, a client-side user found issues my team missed. The incident report, root cause and the process changes that followed are in this portfolio.
- **Team goals I monitor.** The yearly Team Pivot goals for my QA team (FY 2026) and the data I use to track them are on page 20.

Contents

Section	Page	Section	Page
Cover and headline results	1	Jira administration and configuration	13
Profile and capability matrix	2	Jira administration: further configuration	14
Career journey: game QA across platforms and genres	3	Test design and coverage	15
Games shipped: a public release timeline	4	Test scenarios and test cases	16
Types of testing: start to finish	5	Edge-case and negative test design	17
Delivery framework: QA roadmap, stages, entry and exit criteria	6	Requirements traceability (RTM audit)	18
Reporting schedule and risk tracking	7	Team leadership, enablement and research	19
Case study A: casual card-puzzle PC title (2024), results	8	Team Pivot: yearly goals I monitor	20
Case study A: a missed bug, owned and fixed	9	Analytics event validation: schema and QA method	21
Case study A: daily report screenshots	10	Source documents, redacted	22
Case study B: action campaign PC title (2023), release assessment	11	Project management: how a project runs start to finish	23
Bug reporting standard and bug lifecycle	12	What I bring to a new studio, and source index	24

Confidentiality. Client, studio, game and colleague names are anonymised (Project A, Project B). Redacted originals are available to a hiring team on request. Project dates are the real delivery dates.

Profile and capability matrix

I am a Game QA Project Lead who plans, executes and delivers PC game testing end to end, owning both QA and project management across functional, progression, compatibility, interrupt and live-environment cycles. I sit on both sides of the release decision: finding and writing up the defect, and deciding whether the build ships. I administer Jira (workflows, screens, custom fields, permissions) so the data leaving a project is trustworthy, and I am extending my team into automation through proof-of-concept work in Unity and Appium.

Capability	How it shows up in my work	Page
Test strategy and planning	Test plan with scope, exclusions, entry/exit criteria, risks with mitigations, schedule and approvals; QA roadmap and plan set up for client sign-off before a project starts	6-7
Project management and delivery	Schedule, scope, staffing and stakeholder management from kickoff to sign-off on two fixed-deadline PC releases; re-scoping under deadline pressure and staffing changes without losing coverage	6-9
Test design and traceability	Scenarios (6 areas), 204 module-wise test cases, requirements traceability matrix, interrupt and compatibility suites, mobile checklist	15-18
Execution and playtesting	Progression runs in every game mode and difficulty, aggressive/exploratory playtesting, aspect-ratio and resolution sweeps, live-environment testing	8-10
Defect management	Tag + Problem-Action-Location summaries, severity vs priority rules, lifecycle with developer resolutions, triage into P0/P1/P2 buckets	11-12
Reporting and stakeholder communication	19-sheet daily report workbook, weekly status report, incident report, postmortem, closure and handover	7, 9
Jira administration	Work types, workflows and schemes, screens, custom fields, priorities, permission schemes	13-14
Team leadership and goal monitoring	Resource management, test planning, team training, knowledge-transfer sessions, yearly Team Pivot goals tracked against data	19-20
Automation (in progress)	Proof-of-concept level: Unity game automation, Appium on mobile games. Not yet production-scale, and I say so	19

Toolbox

Area	Tools and methods
Planning and tracking	Jira (project lead and administrator), Microsoft Excel (dashboards, pivots, RTM), test plan / procedure / closure templates
Test types	Functional, progression, exploratory and rule-breaking playtesting, compatibility (OS, hardware, resolution, aspect ratio), interrupt, regression, live-build verification, UX observation
Platforms	PC (Windows 10 and 11, low-to-high-end hardware) in depth on Projects A and B; console (PS4, Xbox) and mobile (iOS/Android) production experience from earlier roles (page 3); AR/VR and Web exposure
Learning and POCs	Unity + C#, Appium and Appium Inspector, Selenium WebDriver, TestNG / JUnit, Alt Unity Tester, Android Studio

Projects covered. Project A, casual card-puzzle PC title with a Collector's Edition (Jun-Sep 2024, Project Lead). Project B, action campaign PC title (May-Aug 2023, Jira project lead and QA lead). Both were delivered to external clients from an India-based test team.

Career journey: game QA across platforms and genres

The two deep-dive case studies later in this portfolio (Project A and Project B) are anonymised under client confidentiality. This page uses real employer and project names instead, to show testing depth across console, mobile, PC and emerging platforms that the deep-dives alone do not cover.

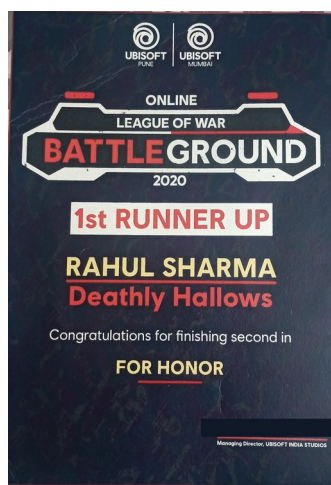
Company	Role	Status	Platform / genre	Highlight
IGT Solutions Pvt Ltd, Gurgaon	Project Lead, Game Testing & QA	Current	PC / Steam — casual, puzzle, strategy, action, iGaming	Leads a QA team of 8+ engineers; shipped 70+ live Steam titles for European and US studios; manages QA for 19+ UAE indie-studio titles per month
Passage.io, USA (Remote)	Senior QA Analyst (Freelance)	Past	PC — Blockchain / NFT metaverse	Shipped Passage World Building, a real-time metaverse title with virtual land, IAP commerce and social collaboration
Passion Gaming Pvt Ltd, Gurgaon	Senior Quality Analyst	Past	PC — real-money iGaming	Led QA for Rummy Passion across Cash, Sit-N-Go, Pool, Points and Deals modes; validated IAP, KYC and tournament back-office flows
Indium Software Pvt Ltd, Bengaluru	Testing Engineer, Level 2	Past	Mobile — PvP/PvE racing, iGaming slots	Owned end-to-end quality for CSR 2 (mobile drag-racing) and iGaming slot titles; found critical crash scenarios through edge-case analysis
Ubisoft Entertainment Pvt Ltd, Pune	Game Tester, QA/QC	Past	PS4 / Xbox / PC — cross-platform	Owned QA delivery for For Honor (hack-and-slash); multiplayer, network and fight-mechanics validation; DLC, VFX/SFX and cinematics testing
U2opia Mobile, Gurgaon	Game Tester (Intern)	Past	Mobile — iOS / Android	Tested 5 mobile titles on Google Play and iOS; tracked KPIs via Firebase, IronSource and Google Analytics
I-Tuple Technologies, Gurgaon	Software QA Analyst (Intern)	Past	Web / API	API testing on Milk Basket, a grocery delivery app, using Swagger and Postman; designed and executed test cases

B.Tech, Computer Science, Manav Rachna International University, Faridabad (2017).

Achievements and certificates



Spot Award for Active Testing Excellence — Indium Software



1st Runner-Up, Online League of War Battleground QA Event 2020 (For Honor) — Ubisoft Entertainment

Games shipped: a public release timeline

Publicly released titles, live today on Steam, mobile app stores and the web. Belote Bistro is the most recent: tested on mobile and released publicly within the last few weeks.

2023 – Present, IGT Solutions Pvt Ltd (PC / Steam, Mobile, Web)

Title	Platform
Belote Bistro (most recent release)	Mobile
Chronicles of Albion — The Magic Convention	PC / Steam
Chronicles of Albion 3	PC / Steam
Jewel Match Origins 3	PC / Steam
Jewel Match Royale Solitaire (3 releases across platforms)	PC / Steam, Mobile
Jewel Match Solitaire Fantasy	PC / Steam
Jewel Match Solitaire L'Amour	PC / Steam
Jewel Match Solitaire Seasons, Standard and Collector's Editions	PC / Steam
Jewel Match Solitaire Winterscape 1, 2 and 3	PC / Steam
Jewel Match Solitaire X	PC / Steam
Jewel Solitaire Tripeaks	Web
JM Solitaire HTML5	Web
Match Venture 4	PC / Steam
Runefall 3	PC / Steam
Solitaire Dreamscapes	PC / Steam
Tales of Lagoon — Orphans of the Ocean	PC / Steam
Zen Quest Garden	PC / Steam

Earlier roles: cross-platform and cross-genre releases

Title	Studio / period	Platform
For Honor	Ubisoft Entertainment, 2019–2021	PS4 / Xbox / PC
CSR 2	Indium Software, 2021–2022	Mobile
Rummy Passion	Passion Gaming, 2022–2023	PC / Web (real-money iGaming)
Passage World Building	Passage.io, 2023	PC (blockchain / NFT metaverse)

Types of testing: start to finish

Each test type checks something different, and each one runs at a different point in the project. Below is the order I follow, from the moment a new build arrives to the moment it goes live.

Stage	Test type	Build / environment	In plain terms
1. New build arrives	Smoke testing	Dev build	A fast, shallow pass to check the build even launches and the core loop works. If smoke testing fails, the build goes back untested — there is no point testing further.
2. Core pass	Functional testing	Dev build	Does each feature do what it is supposed to: buttons, menus, saving, the core mechanics. This is the main body of testing.
3. Core pass	Progression testing	Beta build	Play the game start to finish, in every mode and difficulty, the way a real player would. Confirms the whole journey works, not just individual pieces.
4. Core pass	Exploratory testing	Beta build	Deliberately unscripted: click things in the wrong order, try to break the game the way a curious or impatient player might. Finds the bugs a written test case would never think to check.
5. After every fix	Regression testing	Same build, retested after fix	Re-run the tests around whatever changed, to check the fix did not break something that used to work. This runs continuously, not just once.
6. Before sign-off	Compatibility testing	Release-candidate build	Same build, different hardware, operating system, screen resolution and aspect ratio. Confirms the build behaves consistently across environments, not only on the machine it was built on.
7. Before sign-off	Interrupt testing	Release-candidate build	What happens to the game when a phone call comes in, a notification pops up, the device sleeps, or the charger is plugged in mid-session.
8. Before sign-off	Localization testing	Release-candidate build	Text, dates and currency display correctly in every supported language, with nothing cut off, mistranslated or left in a placeholder state.
9. Before sign-off	Usability / UX testing	Release-candidate build	Is the game intuitive without a manual; where do new players get stuck or confused. Feeds design feedback, not just bug reports.
10. After go-live	Live-build testing	LIVE / production build	One more pass on the real, published build once it is live, to confirm what players are actually getting matches what was signed off.

Regression testing (row 5) is not a one-time stage: it repeats after every fix, all the way through a project, which is why it sits in the middle of this list rather than at the end.

Delivery framework



Figure 1. QA roadmap presented to clients (from the QA Roadmap deck)

Stage	Produced	Checkpoint
1 Initial planning	Test plan, scope, risks	Plan approved by lead, PM and stakeholders
2 Environment setup	Hardware matrix (low to high end), build access, Jira project and permissions	Entry criteria met
3 Test design	Scenarios, cases, RTM, checklists, game guide for self-use	Cases reviewed and approved
4 Test execution	Daily runs on dev, validation and beta builds; progression, compatibility, interrupt	Daily report issued
5 Bug tracking and resolution	Jira lifecycle (incl. Pending Retest), triage meetings, retest and verification	No unverified fixes
6 Test reporting	Daily and weekly reports, dashboards, defect dump	Stakeholders informed
7 Final review and sign-off	Closure report, exit-criteria check, handover documents	Closure report approved
8 Post-release testing	Live-build verification after go-live	Live sign-off

Entry and exit criteria

Entry criteria	Exit criteria
Test environments set up; test plan prepared and approved; Jira project and permissions configured; test cases reviewed and approved; test data and scenarios prepared	All planned test cases executed; all critical and high-priority defects resolved (or waived in writing); test completion report reviewed and approved; test closure report prepared and approved

How I plan the work: Waterfall and Agile together

Two common ways of organising a project: Waterfall does one phase at a time, in order — planning finishes, then design, then execution, then closure — with a review at the end of each. It suits a project that needs one fixed schedule and one fixed budget agreed before work starts. Agile instead works in short, repeated cycles, with regular feedback and room to adjust as new information comes in — it suits work where priorities can shift.

My projects use both, for different parts of the same project. The 8 stages above are Waterfall: one fixed order, approved up front, so the client knows the schedule and budget from day one. The daily work inside stage 4 (test execution) is Agile: a short cycle around each new build, a daily report standing in for a team stand-up, and continuous retesting rather than one long test pass saved for the end. The outer shape is fixed for the client; the inner work stays flexible enough to react to what each new build actually shows.

Reporting schedule and risk tracking

How often I report

Rhythm	Artifact	Purpose
Every session	Daily report workbook (19 sheets): checklist, bug log, mode-by-mode runs, compatibility, observations	Shows what was tested, on which build, OS and hardware, and how long it took
Every build	Retest of fixes plus regression; build version stamped on every log line	Confirms fixes without new breakage
Weekly	Weekly status report: plan, progress, challenges, notes	A written progress record before the next milestone
Per incident	Incident report: timeline, root cause, SLA, impact, corrective action	Names what was missed, why, and what changed afterward
At milestones and closure	Postmortem, closure and handover report, defect dump	Client signs off in writing; the record outlives the project

Known risks and how I handle them (from the test plan)

Risk	Mitigation
Tight deadlines	Prioritise critical tests; risk-based testing
Limited resource availability	Allocate resources by risk; bring in temporary testers if needed
Incomplete requirements	Work with the client to clarify requirements; request the game design document
Tool limitations	Use alternative tools or manual testing where necessary

Crunch time and team churn

- **Crunch (deadline pressure):** re-scope to a risk-based critical path in writing, not silently — the client sees exactly what is deferred and why. Regression testing is never compressed, even when everything else tightens, since a new bug under deadline pressure costs more than a known one. The full step-by-step process, with a worked example from Project A, is on page 8.
- **Churn (testers joining or leaving mid-project):** the documented test cases, scenarios and checklists (pages 15-17) are the actual handover mechanism, not tribal knowledge — a new tester can pick up real coverage within a day. A new tester's first build cycle is paired with an existing one before they run solo, and knowledge-transfer sessions (page 19) cover anything the documents do not.

Scope discipline

The test plan lists backend, database, network and performance testing as out of scope, so the client cannot assume coverage that was never agreed. In practice, interrupt testing (incoming calls, messages, app notifications, PC lock/unlock, battery and charging state, system updates) and basic network-condition checks (using the network simulators set up for the project) were both run and are reflected in the interrupt and compatibility suites on page 9. Lightweight performance signals, such as completion time per level tracked across hardware tiers in the daily report, were also captured, though a full frame-rate or load-testing pass was not. Backend and database testing remained genuinely out of scope, since neither project gave testers server-side or database access.

Roles and responsibilities (from the test plan)

Role	Responsibilities
Project Lead (me)	Daily report, resource management, test planning, test case development, team training, triage, client and stakeholder communication
Test engineers	Test execution and defect logging; retest and verification of fixes
IT support	Set up and maintain the test environment
Development team	Fix identified defects; unit and integration testing at the developer end

Case study A: casual card-puzzle PC title

Platform and environments	PC, Windows 10 and 11, from Core i5 / 8 GB laptops to Ryzen 9 / 32 GB machines. Pre-production, then LIVE in September
Team and effort	Lead plus one QA engineer (2 testers Jun-Aug, 1 in the live phase): 43 days, 576 hours
Scope	Four game modes (Normal, Timed, Hard, Relaxed), map and level progression, shop and power-ups, scene rebuild, bonus-level series, collector extras, settings, exit and interrupts, compatibility
Output	212 defects logged, 31 build versions tested, 204 documented test cases, 197 executed with 186 passed at the July checkpoint (94.4%)

Challenge

A feature-rich edition with four game modes, a long bonus-level series and frequent beta builds, tested by a two-person team. Coverage had to be broad enough to satisfy the client and disciplined enough to be repeatable build after build.

Approach

- **Learn the game first.** Wrote a game guide for self-use before designing tests; test cases are designed around the real mechanics.
- **Plan, then design.** Test plan, test procedure, scenarios, 204 test cases and a requirements matrix.
- **Log everything.** One daily-report workbook: every level played, with mode, difficulty, build, OS, hardware and time to complete; a separate observation sheet for design and UX recommendations.
- **Play it like a player, then like a rule-breaker.** Full progression in each mode, aggressive and exploratory playtesting, bonus-level rounds repeated on new builds (seven rounds), interrupt tests, live-game testing.

Results

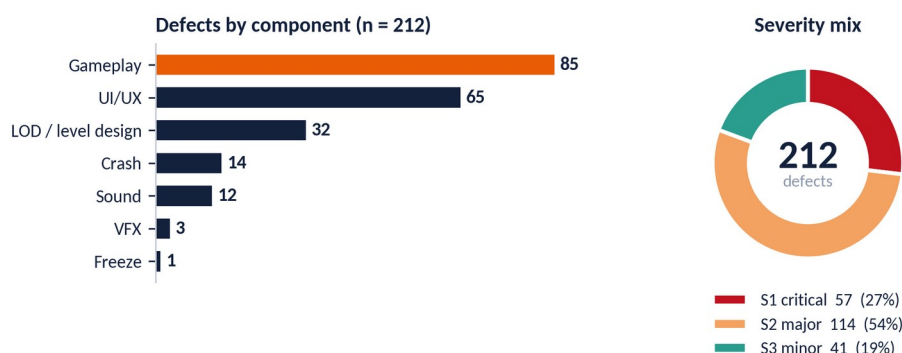


Figure 2. Source: daily report bug log, 212 unique tickets, final recorded severity. S1: 26 gameplay, 17 level-design (LOD/collision), 14 crash.

What the data said. Gameplay and UI/UX account for 150 of 212 defects (71%). More than a quarter of all defects (27%) were S1, and all 14 crash defects (show-stoppers) were among them. The highest-severity risk sat in the flow of the game and in level behaviour rather than in isolated screens, which supports the emphasis on full progression runs and repeated bonus-level rounds.

What I do, step by step, during crunch time

1. **Freeze scope in writing.** Confirm with the client exactly what stays in scope; anything cut is agreed and documented, not silently dropped.
2. **Re-triage by risk, not ticket age.** P0/P1 defects get resourced first; low-severity items get a written waiver, not silence.
3. **Protect regression testing.** Last-minute fixes still get regressed — a new bug under deadline pressure costs more than a known one.
4. **Increase reporting cadence.** Daily moves to per-build or twice-daily, so risk is visible as it builds, not after.
5. **Clear volume before any staffing change.** If the team is set to shrink, close and verify the backlog first, not carry it forward.
6. **Get an explicit, written sign-off.** Closure is a documented decision — fixed, deferred, accepted risk — not just “time ran out.”

Case study A: a missed bug, owned and fixed

The incident

Date	Event	Handling
13 Jul 2024	A client-side user on the beta build reported screen-resolution and aspect-ratio problems, help-file content that no longer matched the game, a misplaced "Scene complete" text at high resolution, overflowing piles in a bonus game and a missing confirmation on "Try Again" (severity S2).	Testers reviewed each report and raised new tickets the same day; escalated to leadership; resolution time recorded as 1 day.
26 Jul 2024	Bonus levels de-synchronised and crashed between the standard and Collector's editions.	Recorded as missed by the team in testing. My analysis: the cross-edition bonus path was not a regular part of the test flow.

Root cause. Recorded: a screen-resolution defect that appears when the player changes resolution while playing. My analysis: the compatibility matrix did not yet include a full resolution and aspect-ratio sweep or the cross-edition bonus path, so a mid-game resolution change was never exercised in standard testing. Impact recorded: medium reputational risk, no penalty, rework required.

Corrective actions and follow-through

- **Recorded action:** a new roadmap, checklist, scenarios and test cases issued as part of the test plan, to be **signed off by the client before a project starts**.
- **By project close:** a dedicated compatibility suite (29 cases), an interrupt suite (7 cases), and resolution / aspect-ratio sweeps logged in the daily report from 27 Aug 2024.
- **Follow-through:** the bonus-level series was re-run in seven rounds on successive beta builds (Jul-Aug), and a live-environment pass followed in September.

Postmortem: interim checkpoint, July 2024

Measure	Result	Reading
Test cases executed / passed / failed	197 / 186 / 11	94.4% pass rate at the checkpoint
Defects logged at checkpoint	114 (S1 31, S2 63, S3 20)	Consistent with the daily bug log (115 by end of July); final count was 212
Recurring themes	Crash, screen resolution, synchronisation	Drove the extra retest and sync coverage

Lessons I carried forward

- **Agree the compatibility matrix up front.** Resolution, aspect ratio and older hardware belong in the plan, not in the postmortem.
- **Ask for the game design document early.** It widens bug coverage and sharpens the test plan; this was the first recommendation in the closure report.
- **Add a low-end performance baseline.** Performance was outside the agreed scope, and the postmortem recommends including it, especially for older devices.
- **Automate the repetitive mechanics.** Documented cases made regression cheap; the next step is automating routine mechanics and interactions.

Case study A: daily report screenshots

Game Checklist												Status					
So.no	Date	Module Name	Build Version	Hours	Minutes	Seconds	OS	Devices	Passed	Failed	Comments						
1	29/8/2024	Game Launch	Beta_2024-08-29-2		30	25	WIN 11	PC	Passed								
2	29/8/2024	Enter/Edit User name	Beta_2024-08-29-2		25	20	WIN 11	PC	Passed								
3	29/8/2024	Game Tutorials	Beta_2024-08-29-2		30	15	WIN 11	PC	Passed								
4	29/8/2024	Setting Wallpaper	Beta_2024-08-29-2		30	40	WIN 11	PC	Passed								
5	29/8/2024	Game Round Win/Lose Prompt	Beta_2024-08-29-2		25	20	WIN 11	PC	Passed								
6	29/8/2024	Stars system	Beta_2024-08-29-2		30	30	WIN 11	PC	Passed								

Figure 3. Game checklist run on each beta build (build version, OS, device, time and result per module)

Gameplay Testing																		
Soak	Date	Level no./Name	Build Version	Difficulty	Game Mode	Hours	Minutes to Complete	Seconds	OS	PC Specifications	Aspect Rat	Screen Resolets	Level Attempts (2,3,X)	Level Stars (1-3)	Solitaire Bonus Level	Trophies Name	Castle Rebuild using Emeralds	Cards Left in Stock
404	17/07/24	396	2024-07-17	Easy	Normal		25	10	WIN 10	Processor-Intel(R) Core(TM) i5-10210U CPU @ 1.60GHz 2.11 GHz RAM-16.0 GB OS- Windows 10 Hard Drive -512GB	16:09	1360 x 768	1	3	0	0	1	4
405	17/07/24	397	2024-07-17	Hard	Normal		30	15	WIN 10	Processor-Intel(R) Core(TM) i5-10210U CPU @ 1.60GHz 2.11 GHz RAM-16.0 GB OS- Windows 10 Hard Drive -512GB	5:04	1280 x 1024	3	3	0	0	1	0
406	17/07/24	398	2024-07-17	Easy	Normal		25	25	WIN 10	Processor-Intel(R) Core(TM) i5-10210U CPU @ 1.60GHz 2.11 GHz RAM-16.0 GB OS- Windows 10 Hard Drive -512GB	16:10	1280 x 800	1	3	0	0	1	0
407	17/07/24	399	2024-07-17	Medium	Normal		25	30	WIN 10	Processor-Intel(R) Core(TM) i5-10210U CPU @ 1.60GHz 2.11 GHz RAM-16.0 GB OS- Windows 10 Hard Drive -512GB	5:03	1280 x 768	1	3	0	0	1	0
408	17/07/24	400	2024-07-17	Medium	Normal		25	45	WIN 10	Processor-Intel(R) Core(TM) i5-10210U CPU @ 1.60GHz 2.11 GHz RAM-16.0 GB OS- Windows 10 Hard Drive -512GB	16:09	1280 x 720	1	3	0	0	1	0
													550	1163	0	0	292	1979

Figure 4. Progression log: level, difficulty, mode, hardware, aspect ratio, resolution, attempts, stars and cards left

Coverage snapshot

Dimension	Covered
Game modes	Normal, Timed, Hard, Relaxed, each with its own run sheet; extra emerald-play runs
Bonus levels	Seven rounds across successive beta builds, easy to hard difficulty sets
Display	Aspect ratios 4:3, 16:9, 16:10; resolutions from 800x900 to 1600x900
Systems	Windows 10 and 11; laptops and desktops from Core i5 / 8 GB to Ryzen 9 / 32 GB
Environments	Pre-production builds (Jun-Aug) and LIVE (Sep)

Case study B: action campaign PC title

152 defects, 5 test cycles	59% closed (90 of 152)	15 / 24 Highest and High still unresolved	0 / 5 crash defects open	84% of defects with one developer
-------------------------------	---------------------------	--	-----------------------------	--------------------------------------

Verdict: not ready for release sign-off. Stability is good (all 5 crashes and every blocker closed). The risk is functional and concentrated in late-game boss and AI behaviour, level-progression exploits and save-data integrity. The newest build had 32 of its 36 defects unresolved, so its fix and verify phase had not started.

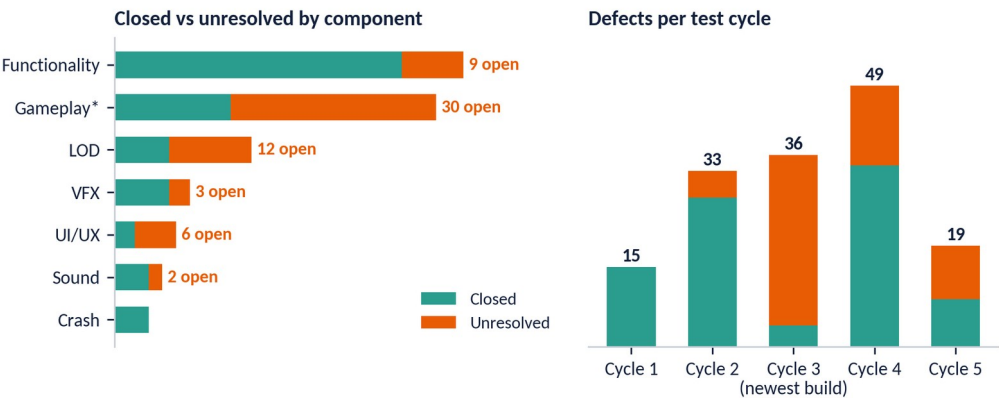


Figure 5. *Includes 3C-Gameplay. Cycles ordered by first defect logged; Cycle 3 is the newest build. Two defects with no component or an installation tag are not shown.

Exit-criteria check

Criterion	Status	Data
Zero open Highest defects	FAIL	2 open (one reopened)
No open High defects, or an agreed waiver	FAIL	13 open, mostly gameplay and AI
No open crash or blocker defects	PASS	All crash defects closed
At least 90% of defects closed	FAIL	59.2% closed
No reopened defects left unverified	FAIL	6 reopened
Test-case pass rate reported	NOT ASSESSABLE	Jira export holds defects, not test-case results

Thresholds are common industry defaults, flagged in the report as illustrative until replaced by the project's agreed exit criteria.

Triage decisions

Bucket	Scope	Action
P0: show-stoppers, fix now	4 defects: post-level flow sends the player to the start page (reopened); save data destroyed on new game; two level-skip exploits	Fix and verify before the next candidate build; add regression for progression and save data
P1: boss and AI combat	9 defects: idle bots, boss standing still, enemies dying without attack, cannons reactivating after checkpoint reload	Find the shared enemy-AI state root cause; likely few fixes close many tickets
P2: reopened and open backlog	5 reopened, 8 open or new	Re-verify why fixes failed retest; assign owners; group 11 collision/LOD issues into one pass

Process findings beyond the bugs: one developer held 127 of 152 defects and all 62 unresolved ones (single point of failure); 82% of defects sat at Medium priority; 84 of 90 closed defects had no resolution value; one duplicate, one missing component and two feature requests filed as bugs. These map to the Jira configuration on page 13.

Bug reporting standard and bug lifecycle

The bug report standard

Field	Standard
Summary	[TAG] + Problem-Action-Location. Example shape: [Gameplay] Enemy does not attack when the player strikes from behind in level X
Steps to reproduce	Prerequisites plus numbered steps, including what the tester did to isolate the issue
Description	What happened and when; system specification; expected and actual (observed) behaviour, kept concise
Severity	By effect on the game. S1 crash, stuck or freeze (show-stopper); S2 gameplay feature or mechanic; S3 UI/UX and localisation
Priority	By urgency of the fix: Highest, High, Medium, Low
Component and labels	Issue-specific only; platform tag when relevant
Attachments	Screenshots and video named with the ticket ID, for example ID_1, ID_2

Tag taxonomy: Gameplay (mechanics, AI, exploits), LOD (collision, ragdoll, out-of-map, missing textures), Sound/SFX, LOC (localisation), Crash/Error/Freeze, VFX, UI/UX, Network/Online, 3C (camera, collision, character), Leaderboard, plus platform tags PC, Xbox, PS5, Android, iOS.

Bug lifecycle and developer resolutions

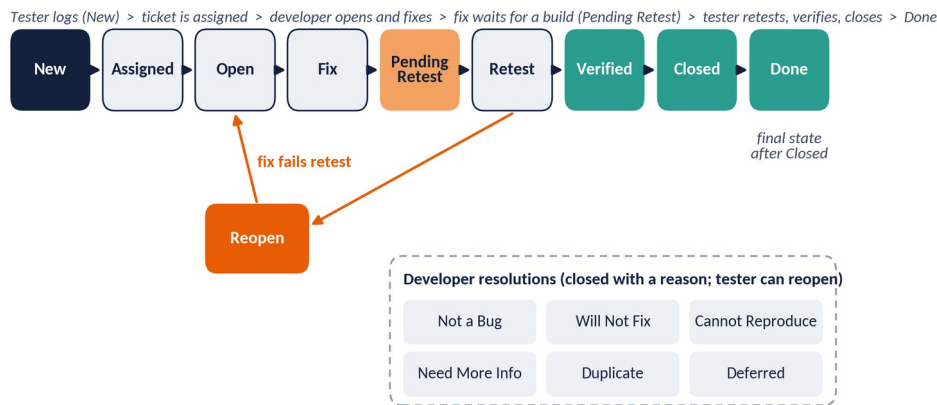


Figure 6. Current lifecycle: New, Assigned, Open, Fix, Pending Retest, Retest, Verified, Closed, Done. A failed retest goes to Reopen. Resolutions close a ticket with a reason; the tester can reopen any of them if the reasoning does not hold.

Status	Meaning	Owner
Assigned	Triaged and handed to a developer; work has not started yet	Developer (queue)
Pending Retest	Fix delivered by the developer; waiting for a build the tester can verify on	Tester (queue)
Retest	Tester is actively verifying the fix on the build	Tester
Closed / Done	Verified and closed; Done is the final state after Closed	Tester

Why it matters. Project B’s export showed what happens without one lifecycle: free-text reproducibility in mixed formats, empty resolutions, and 47 of 62 unresolved defects sitting at Assigned — the single largest bucket, larger than Open itself. Tracking Assigned and Pending Retest as distinct states makes both ends of the backlog visible before they become a release risk.

How we run triage

Triage is a standing meeting, not an ad hoc conversation: it ran daily during active cycles, with the lead, test engineers and, where available, a developer. Every bug leaves with a severity (tester-set, by effect), a priority (set jointly, by urgency) and an owner — kept separate on purpose, since a low-severity bug can still be high priority against a deadline.

- **Worked example:** the P0/P1/P2 buckets on page 11 are triage output — four defects bucketed P0 because they blocked progression or corrupted save data, nine bucketed P1 for sharing one root cause, the rest re-verified or reassigned in P2.

Jira administration and configuration

Area	What I configured
Work types and schemes	Bug, Epic and other issue types; which types each project can use
Workflows and schemes	Custom bug workflow with the lifecycle statuses on page 12; different workflows per issue type; workflow schemes shared across projects
Screens and screen schemes	Which fields appear when logging a Bug versus an Epic; adding and removing sections per issue type
Custom fields and field configuration	Custom fields (the export shows Reproducibility, Steps to Reproduce and Type) created, added to the Bug screen and made compulsory
Priorities and priority schemes	Priority levels created; priority schemes applied per project
Permission schemes	Group-based and project-based access with default settings for new projects

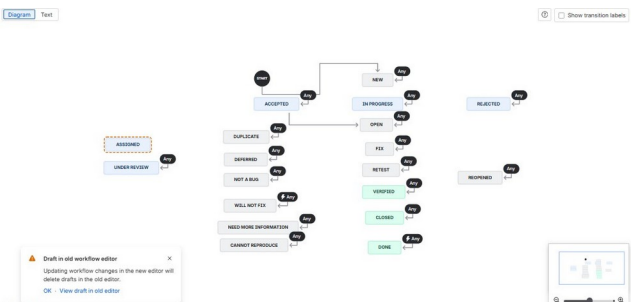


Figure 7. Bug workflow as built in Jira admin. This earlier build predates the Pending Retest step described on page 12.

Component	Description	Component lead
LOD	Level details/design issue related to the collision, Ragdoll, going out of the map. Texture missing	rahul sharma
VFX	Animation or any kind of presentation bug like illumination or Bugs affecting the visual elements of the game, such as graphical glitches, rendering issues, or visual artefacts	rahul sharma
Sound	Bugs related to sound effects, music, voice-overs, or any audio-related problems, such as missing or distorted audio.	rahul sharma
3C- Gameplay	It includes Camera Collision and Character. Issues related to user/AI camera rotation, user props glitches and collision with the environment while using emotos or any feat	rahul sharma
Gameplay	Bugs affecting the core gameplay mechanics, rules, or interactions within the game, including gameplay imbalances, AI behaviour, or glitches during gameplay	rahul sharma
Crash	Bugs that cause the game to crash, freeze, or display error messages, preventing users from playing or progressing	rahul sharma
UI/UX	Problems related to the user interface (UI) or user experience (UX), such as incorrect layout, misaligned elements, broken links, or usability issues	rahul sharma
Localization	Problems with the game's translation or localization, such as incorrect text, missing translations, or cultural inconsistencies	rahul sharma
Online	Bugs occurring in multiplayer or online features, including connection problems, synchronization issues, lag, or disconnections	rahul sharma
Leaderboard	Incorrect scoring	rahul sharma

Figure 8. Component list with definitions used for triage. Each component has an owner, so a new bug routes to the right person immediately.

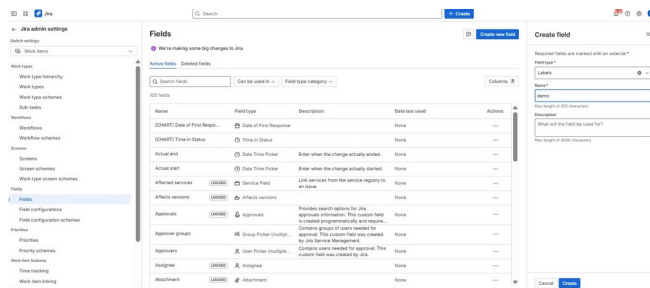


Figure 9. Creating a custom field. The same screen is used to add real fields such as Reproducibility and Severity to the Bug screen.

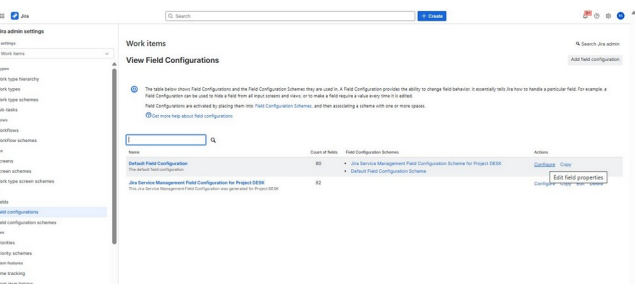


Figure 10. Field configuration. Toggling Required here stops a bug being saved without that field filled in.

Jira administration: further configuration

The previous page listed what I configure. These screenshots are pulled directly from the Jira admin console I administered.

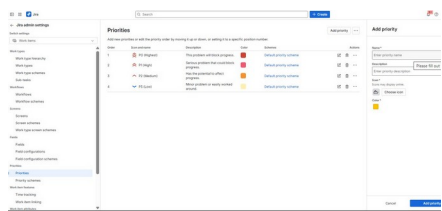


Figure 15. Priorities created with descriptions and colour coding. Priority drives triage order and is kept separate from severity.

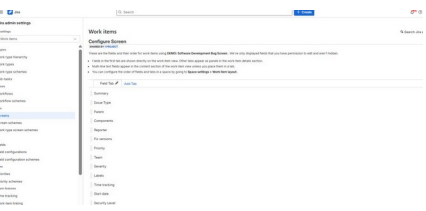


Figure 16. Screen configuration: fields and order shown on the Bug screen. This is exactly what a tester sees while logging a bug.

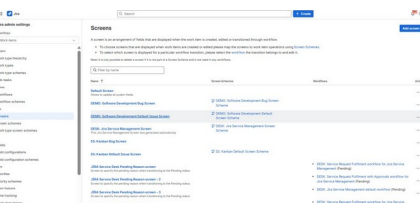


Figure 17. Screens list: a separate screen per work item type, so a Bug screen never shows Epic-only fields.

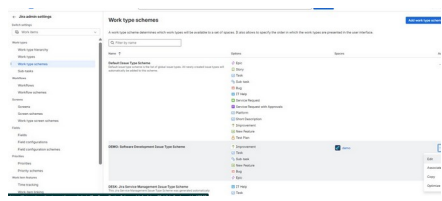


Figure 18. Work type schemes: which issue types (Bug, Epic, Task) a given project is allowed to create.

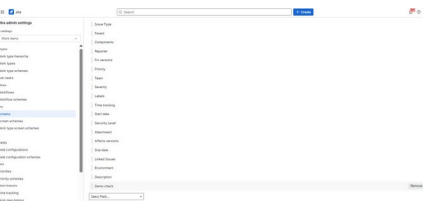


Figure 19. The full field list on the Bug screen, including a custom field (Demo check) added and ready to use.

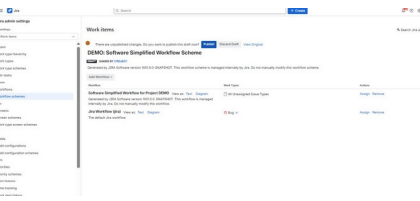


Figure 20. Workflow scheme: a workflow assigned per work type, so Bug and Epic can follow different lifecycles in one project.

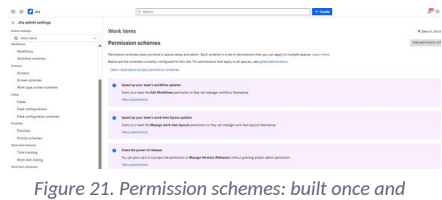


Figure 21. Permission schemes: built once and reused, so a new project inherits the right access from day one.

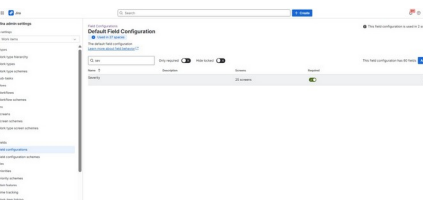


Figure 22. Making Severity compulsory in the field configuration, so a bug cannot be logged without one.

Screenshots are cropped to remove project, game and colleague names; only generic Jira admin structure is shown.

Test design and coverage

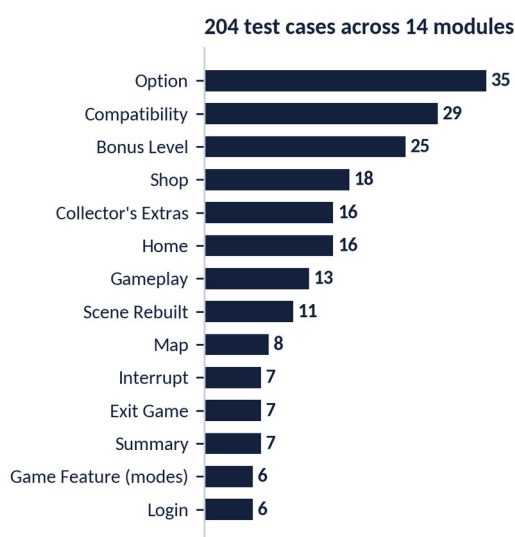


Figure 11. 204 test cases across 14 modules

What each artifact shows

- **Test scenarios** (6 areas): installation and uninstallation, functional, compatibility, localisation, GUI, interrupt.
- **Test cases** (204, 14 modules): pre-requisites, steps, expected and actual result, status, comments.
- **Checklist**: 53 module-level checks logged against build versions in the daily report.
- **Mobile checklist**: 68 checks across device, network, app and UI, referencing Android and Apple platform guidelines.
- **Test procedure**: step-by-step, per feature, with roles for logging, triage and reporting.

The original RTM, before my audit

Req. Description					
A	B	C	D	E	F
Req. ID	Req. Description	Module Name	Test case ID	Test Cases Description	Status
FR1.1	The game shall draw a card from a stack on the board.	Gameplay	TC_02	Verify that the user can draw the card from the stack.	Pending
FR1.2	The game shall display different types of cards.	Gameplay	TC_03	Verify that the user can select the matching card from the board.	Pending
FR1.3	The game shall be available in different Mode.	Gameplay Mode	TC_02, TC_03, TC 04	Verify that the user has initiated gameplay in Normal, Timed, and Hard Mode.	Pending
FR2.1	The system shall allow players to select the cards from the board.	Gameplay	TC_03	Verify that the user can select the matching card from the board.	Pending
FR2.2	The system shall decrease the card count after using undo power up.	Gameplay	TC_05	Verify that the count of cards decreases after using undo power up	Pending
FR3.1	The system shall display the Total gold coins during gameplay.	Gameplay	TC_08	Verify that the gold coin from the board is added to the total gold coin in the HUD.	Pending
FR3.2	The system shall award Emeralds for clearing the board for all matching cards.	Gameplay	TC_08	Verify that the count is displayed in the HUD after collecting emeralds from the board.	Pending
FR3.3	The game shall award Trophies.	Gameplay	TC_11	Verify that the user is receiving Trophies and coins in between the game	Pending
FR4.1	The game shall have various levels.	Map	TC_03	Verify that the user can see current level number at the top of the map.	Pending

Figure 12. Original requirements traceability matrix: requirement, module, test case IDs, description, status

What the audit found (detail on page 18): several requirements pointed at the wrong test case; test-case IDs repeat across sheets (every sheet has its own TC_01); three requirements were carried over from a mobile project into a PC-only scope; and every status read Pending.

Test scenarios and test cases

These are the two working documents that everything on the previous page was counted from: the scenario workbook (6 areas, one tab per area) and the module-wise test case workbook (204 cases across 14 modules). Both are shown here with the real game title replaced.

Test scenarios: Installation and Login modules

Credits	
Prepared By:	Rahul Sharma

Test Scenario ID	Module	Pre - Requisite	Test Step	Test Description	Test Designer
Installation					
GT_1.1	Availability	1. User should have PC 2. Internet Connectivity	1. The exe file of the the game Collector's Edition" Game should be available on the PC.	Verify that the exe file of the the game Collector's Edition" Game should be available on the PC.	Rahul Sharma
GT_1.2	Error Free	1. User should have PC 2. Internet Connectivity	1. It should be download and installed without any error	Verify that the game download and install without any error	Rahul Sharma
GT_1.3	Play Offline	1. User should have PC 2. Internet Connectivity	1. User should able to play Offline	Verify that the User is able to play the game offline.	Rahul Sharma
GT_1.4	Update	1. User should have PC 2. Internet Connectivity	1. User should be able Update the game if required.	Verify that User is able Update the game if required.	Rahul Sharma
GT_1.5	Permission	1. User should have PC 2. Internet Connectivity	1. Verify that user allowed the Permission to access	Verify that user allowed the Permission to access.	Rahul Sharma
Login					
GT_2.1	New Player Registration	1. User should have the game Collector's Edition" Game installed on the PC.	1. Launch the the game Collector's Edition" Game on the PC. 2. Navigate to "User" Screen. 3. Enter Username. 4. Click on "Ok" Button	Verify that user is able to create new user using username	Rahul Sharma

Figure 12a. Test Scenarios workbook: Installation module (5 scenarios) and the start of Login, with pre-requisite, test step and description columns.

Test cases: Gameplay module

TestCase_ID	Test Name	Pre-Requisite	Test Steps	Test Description
TC_01	Gameplay	1. User should have the game Collector's Edition" Game installed on the PC. 2. The player has been successfully registered in the game.	1. Launch the the game Collector's Edition" Game on the PC. 2. Click on Play Button. 3. Select the level to play. 4. Double-click on the level or click on the 'Play' button.	Verify that user is able to navigate the selected level.
TC_02	Draw card from stack	1. User should have the game Collector's Edition" Game installed on the PC. 2. The player has been successfully registered in the game.	1. Launch the the game Collector's Edition" Game on the PC. 2. Click on Play Button. 3. Select level to play. 4. Click on card stack.	Verify that the user can draw the card from the stack.
TC_03	Select card from board	1. User should have the game Collector's Edition" Game installed on the PC. 2. The player has been successfully	1. Launch the the game Collector's Edition" Game on the PC. 2. Click on Play Button. 3. Select level to play	Verify that the user can select the matching card from the board.

Figure 12b. Test Cases workbook: the first six cases of the Gameplay module (TC_01-TC_06), in the pre-requisite / steps / expected-result format used across all 14 modules.

Both workbooks use the same structure throughout: a module cover block naming the areas covered and who prepared it, then one row per scenario or case. The game title is replaced with "the game Collector's Edition" throughout; everything else, including the wording of each step, is unchanged.

Edge-case and negative test design

Documented test cases confirm a feature works when used as intended. Edge cases check where a feature is likely to break instead: boundary values, rapid or conflicting input, and the network or app-state conditions a player will eventually hit by accident. Two excerpts below, redacted, from the edge-case workbooks I maintain alongside the main test-case set.

Common edge cases: network, input and account state

A1			
Sr. No			
A	B	C	D
Sr. No	Module Name	Test Description	Expected Result
TC_1	Offline Launch	Verify that the game does not remain stuck on the loading screen when launched without internet connectivity during first-time installation.	The game should display an appropriate network error message or switch to offline mode without crashing.
TC_2	Start Button Response	Verify that rapidly tapping the Play/Start button multiple times during initial loading does not open duplicate gameplay sessions.	Only one game session should start and duplicate loading screens should not appear.
TC_3	Loading Recovery	Verify that minimizing the game at different loading percentages does not restart or freeze the loading process.	The game should resume loading correctly without freezing or restarting unexpectedly.
TC_4	Login Validation	Verify that entering incorrect credentials repeatedly with invalid formats and special characters does not break login validation.	The game should display validation messages without crashing or freezing.
TC_5	Account Synchronization	Verify that account synchronization continues safely when internet connectivity fluctuates repeatedly during sync	The sync process should retry or restore data safely

Figure 34. Offline launch, rapid input, background/resume and account sync, the kind of edge case that applies to almost any PC or mobile title.

Genre-specific edge cases: a tower-defense example

TestCase ID	Module Name	Pre-Requisite	Test Step	Test Description
TC_01	Core Health Boundary	1. The user should have downloaded and installed the tower-defense title from Steam on the PC.	1. Launch the tower-defense title on the PC. 2. Click on the 'Play' button. 3. Select any city. 4. Start and play any level. 5. Allow several enemies to attack the nano core simultaneously	Verify game behaviour when nano core HP reaches exactly 0 during simultaneous attacks
TC_02	Enemy Damage Count	1. The user should have downloaded and installed the tower-defense title from Steam on the PC.	1. Launch the tower-defense title on the PC. 2. Click on the 'Play' button. 3. Select any city. 4. Start and play any level. 5. Spawn a heavy enemy wave and allow all enemies to attack the core together	Verify that enemy damage is calculated accurately when many enemies attack together. Ensure HP reduction does not skip, freeze, or reduce incorrectly.
TC_03	Portal Spawn	1. The user should have downloaded and installed the tower-defense title from Steam on the PC.	1. Launch the tower-defense title on the PC. 2. Click on the 'Play' button. 3. Select any city. 4. Start and play any level. 5. Allow several active portals at the same time	Verify that enemies spawn correctly from multiple portals without disappearing, overlapping, or getting stuck at spawn points
TC_04	Wave Indicator	1. The user should have downloaded and installed the tower-defense title from Steam on the PC.	1. Launch the tower-defense title on the PC. 2. Click on the 'Play' button. 3. Select any city. 4. Start and play any level. 5. Monitor the wave bar continuously with the coming enemy	Verify that the wave indicator remains synchronized with actual enemy spawn timing
		1. The user should have downloaded	1. Launch the tower-defense title on the PC. 2. Click on the 'Play' button.	

Figure 35. Boundary and concurrency cases from a separate tower-defense engagement: core HP at exactly zero, simultaneous multi-portal spawns, wave-indicator sync under load.

Both workbooks follow the same shape as the main test-case set (pre-requisite, steps, expected result) so they slot into the same triage and regression process, rather than living as a separate, easily-forgotten list.

Requirements traceability (RTM audit)

I re-checked every mapping in the original matrix. The corrected view uses module-prefixed IDs (for example GP-TC02, SHOP-TC05) and classifies each requirement by how directly it is tested. Statuses show coverage, not execution results, and are my classification of the workbook.

Req.	Requirement (short)	Corrected test link	Coverage
FR1.1	Draw a card from the stack	GP-TC02 Draw card from stack	Direct
FR1.3	Game available in different modes	GF-TC01, 03, 05, 06 (Relaxed, Timed, Normal, Hard)	Direct
FR2.2	Undo reduces the undo count	GP-TC04 Undo (was mapped to TC_05)	Direct
FR3.1	Show total gold coins	GP-TC09 Total gold coin (was TC_08)	Direct
FR4.2	Next level unlocks on completion	SUM-TC07 Continue (was Map TC_02, Stars)	Partial
FR5.2	Buy power-ups during gameplay	SHOP-TC02 to TC15 (tested from the shop page)	Partial
FR1.2	Display different card types	No dedicated case	Gap
NFR3.1	Progress saved automatically	No dedicated case (was mapped to a Play test)	Gap
NFR4.2	Layout adapts to landscape screen sizes	Re-scoped: COMPAT-TC01 to TC29 (resolutions, aspect ratios)	Direct
NFR1.1, NFR4.1	Load time on mobile; available on Android	Out of scope for a PC title	N/A

Representative subset shown. All 25 requirements were re-read against the test-case workbook.

15	5	3	2
Direct (incl. one re-scoped to PC)	Partial	Gap	Mobile carry-over, N/A

Finding that mattered. The unlock rule for the next level had no dedicated case, and three of the first nine defects logged on the project (in the first two days) were level-unlock errors. Coverage gaps and defects clustered in the same place, which is the argument for keeping the RTM alive rather than filing it once.

What I improved

- Every requirement mapped to a module-prefixed test-case ID, so IDs are unique across sheets.
- Two mobile-only requirements marked out of scope for the PC title; one re-scoped to the compatibility suite.
- Three gaps raised as new test cases to write: card types, auto-save, and next-level unlock rule.

Team leadership, enablement and research

Team size and rhythm: Project A ran with a lead plus one QA engineer (two testers from June to August); Project B had three QA reporters plus the lead. Weekly status reports carried plan, progress, challenges and risks to the client, and the team held knowledge-transfer sessions on new defect classes such as game exploits.

Weekly status report: example

WSR- Gaming Project

2. Current and Next Week Plan

Sequence Number	Summary	Task/Remark	Priority	Status
Project 1	XXX Project: a) Regression testing is in progress. b) Recent Fixed Blocker retesting has been completed for the current build. c) Few defect fixed (~6) by developer. d) Latest defect list shared with stakeholders	Task	P1	Completed

Figure 23. Current and next-week plan: task, priority and % complete tracked weekly

Sequence Number	Details	Status	Priority
1	Automation solutions are different for each Game type. Need to learn different Automation frameworks and tools available in the market. Mobile game testing – Appium Web apps – Selenium High Budget Games (AAA) – Unreal Engine and Unity Need to do more R&D on Game Automation testing frameworks on different platforms	In progress	P2

Figure 24. Challenges section: automation tooling gaps raised and tracked to resolution

Enablement and automation R&D

- **Team learning plan:** Selenium WebDriver, Appium, TestNG / JUnit, C#, Alt Unity Tester, Android Studio and Appium Inspector, plus preparation for the ISTQB game-testing syllabus.
- **Proofs of concept:** automation of a small Unity game and of Android mobile games; a master framework adapted for web and mobile game automation. These are POC-stage and I describe them that way.
- **Key learning:** automation differs by game type (Appium for mobile, Selenium for web, Unity or Unreal tooling for AAA), so tool choice starts from the game, not the other way round.
- **Playtesting range:** hands-on MMORPG playtesting and open-source game testing to build genre knowledge before client work.

Research and knowledge assets

Asset	What it contains	Use in a project
Game bug taxonomy	15 bug classes (gameplay, collision, visual, audio, UI, save/load, performance, compatibility, localisation, quest logic, memory leaks, network, exploits, crashes, balance) and 27 game areas to check	Shapes test design and Jira tags
Mobile game testing phases	Device performance, network performance and API/server performance; interrupts, battery, background state, jitter, network switching	Seeds the mobile checklist
Test-tool assessment	Four frameworks (T-Plan Robot, Robotium, Calabash, Appium) compared on platform support, Jira and CI integration, reporting, licence and more	Tool selection; Appium is the one explored in POCs
Testing mindset and techniques	Five ways to hunt bugs (follow rules, break rules, unconventional, use everything, extreme inputs) and 12 techniques (combinatorial, soak, recovery, play testing and others)	Briefs new testers before their first exploratory pass
Analytics validation	Event/attribute schema and QA checks; see page 21 for the corrected version and worked example	Verifies telemetry before release
Help-text and content review	Rule descriptions and help text checked against actual game behaviour across 11 solitaire variants	Caught rule and wording mismatches
E-sports technology brief	Landscape of 16 esports solution categories, from hardware to anti-cheat	Background reading, not yet applied on a live project

Several notes are compiled from public sources and are offered as reference material, not original research. Robotium and Calabash are largely legacy today, so the tool assessment is best read as a method for choosing tools, not a current recommendation.

Team Pivot: yearly goals I monitor



Figure 13. Weightage by pillar (sums to 100%): Performance 30, Initiative 20, Value 30, Outreach 10, Talent 10

Acronym	Category	Focus Area	Metric	Weightage
P	Performance	Bug Identification & Reporting	Identify and report high-quality functional, gameplay, UI/UX, and performance bugs with detailed reproduction steps, screenshots/videos, and proper severity/priority. Maintain 95%+ defect accuracy with minimal bug rejections.	10%
		Game Testing Accuracy & Test Coverage	Perform comprehensive testing PC/Mobile games by covering gameplay, menus, levels, features, edge cases, and platform compatibility to achieve 90%+ test coverage.	10%
		Usability/UX Feedback and Functional & Regression Testing	Execute Functional, Regression, Smoke, and Exploratory Testing. Provide actionable UI/UX and gameplay improvement suggestions through structured Excel reports for every assigned game/build.	10%
I	Initiative	Test Process Improvement	Create and enhance reusable test checklists, regression suites, smoke testing checklists, and reporting templates to improve testing efficiency and consistency.	5%
		Game Progression Initiative	Complete game progression by exploring all available features, levels, game modes, achievements, and hidden scenarios to maximise defect discovery and gameplay validation.	10%
		Gameplay Experience Suggestion	Provide practical gameplay balancing, UI/UX, monetization, onboarding, and quality improvement suggestions for every evaluated game.	5%
V	Value – Financials	Productivity /Utilization	100%	10.0%
		Quality - Defect Leakage to UAT/Production	100%	15.0%
		Process Adherence (Defects/Productivity/Timesheets/Efforts)	100%	5.0%
O	Outreach – Partnerships	Stakeholder feedback	Collaborate effectively with Project Leads, QA members, by sharing testing observations, validating fixes, and providing daily bug status reports (Open,	10%

Figure 14. Team Pivot framework as issued (screenshot of the yearly goals workbook; Talent pillar is further down the sheet)

Pillar	Focus area (weight)	How I monitor it	Shown in this portfolio
P Performance 30%	Bug identification and reporting (10%)	Review logged bugs against the reporting standard; watch rejected, duplicate and Not-a-Bug resolutions per tester	Page 12; Jira exports
	Test accuracy and coverage (10%)	RTM and test-case coverage reviews (Direct, Partial, Gap)	Page 18
	Usability/UX feedback, functional and regression testing (10%)	Structured Excel reports per game and build; observation sheet	Pages 8, 10
I Initiative 20%	Test process improvement (5%)	Reusable checklists, suites, templates reviewed after each project	Pages 9, 15-16
	Game progression initiative (10%)	Level-by-level progression log across modes and difficulties	Page 10
	Gameplay experience suggestions (5%)	Observation sheet: balancing, UI/UX, onboarding, monetisation ideas	Page 8
V Value 30%	Productivity / utilisation (10%)	Time logged per module and level in the daily report	Pages 8, 10
	Defect leakage to UAT/production (15%)	Escapes tracked through incident reports and postmortem	Page 9
	Process adherence (5%)	Mandatory Jira fields; timesheet and effort entries checked	Pages 12, 13
O Outreach 10%	Stakeholder feedback (10%)	Daily bug status reports (Open, Closed, Reopened, New); weekly report	Pages 7, 19
T Talent 10%	Annual certifications (5%): ISTQB QA, N8N AI	Goal set for the year; status tracked per tester	Page 19
	Career development (5%)	Learning plan in game QA, exploratory testing, defect analysis, automation basics	Page 19

Weights and focus areas are taken from the yearly goals workbook. Scores are not shown here; results are shared with the hiring team on request.

Analytics event validation: schema and QA method

Part of release sign-off is confirming that a game's own analytics are trustworthy: that progression, monetisation and error events actually fire, with the fields a product or LiveOps team will query later. The table below is the event/attribute schema I check against, generalised from Unity's event categories (Business, Ad Resource, Progression, Error, Design, Custom).

Event type	Event	Attributes checked in QA
Progression	Level Started	levelNumber, levelDifficulty, startTime
Progression	Level Completed	levelNumber, levelDifficulty, completeTime, starsEarned
Business	Purchase / IAP	productId, purchaseAmount, currency, status
Ad Resource	Ad Loaded / Started / Completed / Closed	adUnitId, timestamp, status
Ad Reward	Ad Reward granted	adUnitId, rewardType, rewardAmount, status
Error	Error Occurred / Crash	errorId or crashId, message or log
Design	Custom Event	customEventName, customEventAttributes

Figure 33. Reference schema used to brief testers on what each event should carry before a build is verified.

How I verified this example

My working notes included a Unity code sample for a rewarded-ad callback. On review, its field names (for example `rewardedAdPlacement`, `rewardedAdStartDateTime`) do not match any Unity Ads API I can verify, so the sample was not reliable as a real integration reference. I have not reproduced that code here: I would rather show a corrected, honestly-scope schema than a plausible-looking snippet I cannot stand behind.

What I actually check in QA

- **Functional verification (positive test case):** trigger the action (complete a level, watch a rewarded ad, hit an error state) and confirm the event fires and appears in the analytics dashboard or debug console within an expected window.
- **Data validation:** each event's payload is checked field by field against the schema; for example a Level Completed event must carry the level number, difficulty and stars earned, not just a bare timestamp.
- **Negative and duplicate-event testing:** confirm the event fires exactly once, not zero times (a missed event) and not twice (a duplicate). Both are common regressions after a refactor, so this runs as part of every regression cycle, not as a one-off check.
- **Interrupt testing:** confirm an event triggered just before a call, notification or app backgrounding still reports correctly, consistent with the interrupt testing defined on page 5.
- **Boundary value analysis:** check numeric fields at their edges, not just typical values — level 1 and the final level, zero stars and the maximum stars, a zero-value purchase and the highest-priced item. Off-by-one errors in event data usually show up at these edges, not in the middle of the range.
- **Exploratory testing:** go off-script and try to fire events in unexpected combinations or orders, for example backing out mid-purchase or completing a level twice in quick succession, to see whether the event still logs cleanly or logs something misleading.
- **Progression testing:** play a full level sequence start to finish and confirm every Level Started / Level Completed pair fires in the right order, rather than checking the first and last level in isolation.

Source documents, redacted

Every claim earlier in this portfolio traces to one of these documents. Screenshots below are taken directly from the real files, with the game title, studio name, ticket-key prefixes, level names and the one developer name redacted.

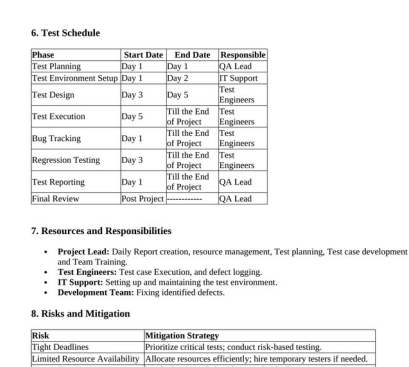


Figure 25. Test Plan: schedule and risk register

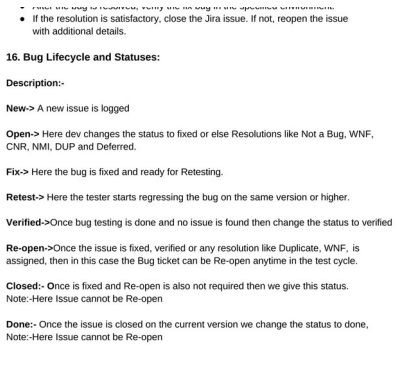
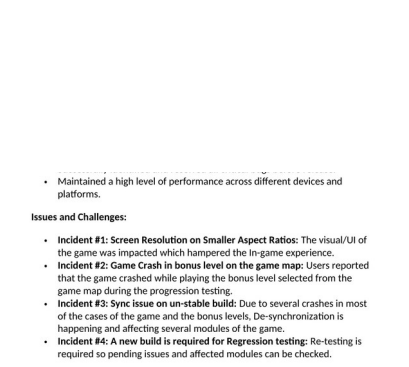


Figure 26. Bug Reporting SOP: the lifecycle statuses as originally written

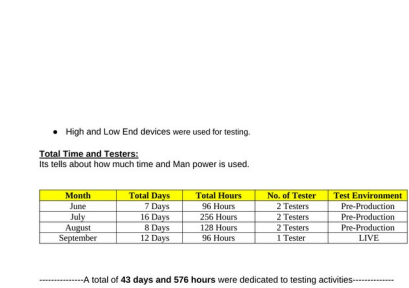


Figure 27. Incident Report: the 13-26 Jul 2024 event log

Figure 28. Postmortem: issues and root-cause analysis



Figure 30. Bug Triage & Test Summary: executive summary (Project B)

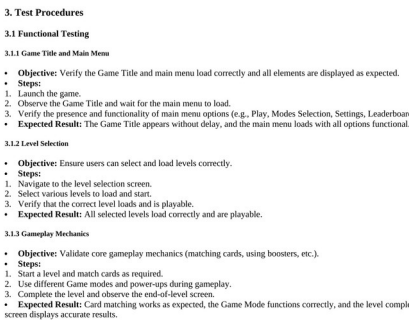


Figure 31. Testing Procedure: objective, steps and expected result per feature

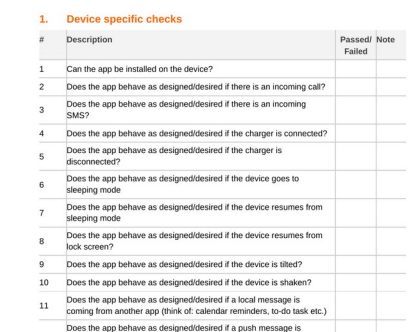


Figure 32. Mobile checklist: device-specific checks

Two source tables are deliberately left out here: the Closure Report's own bug-severity count (S1 62/S2 92/S3 58) disagrees with the daily-log figures used on page 8 (S1 57/S2 114/S3 41); this portfolio uses the log because it reconciles to individual tickets. The Postmortem's "97% coverage / zero critical bugs" claim also sits unreconciled against 31 S1 bugs logged in the same document, so it is not reproduced as a claim anywhere in this portfolio.

Project management: how a project runs start to finish

Every case study in this portfolio is also a project management record: two fixed-deadline client engagements I ran from initiation through to closure. This page sets out that discipline directly, drawing on the sources presented in full elsewhere.

Discipline	What I actually did	Source
Project delivery	Staged delivery from initiation to closure against the 8-stage roadmap, with a written plan, entry/exit criteria, and a closure report signed off at the end of each engagement	Pages 6, 7
Client and stakeholder communication	Weekly status reports; incidents escalated to leadership and reported to the client within 1 day of discovery; closure signed off in writing, not assumed	Page 7
Resource allocation	Headcount matched to project phase, not fixed for the whole project: two testers through pre-production, scaled to one for the September live-monitoring phase; three reporters plus the lead on Project B	Pages 7, 8
People management	Daily task allocation across the team; each new tester's first build cycle paired with an experienced one before working solo; knowledge-transfer sessions run whenever a new defect class emerged	Pages 7, 19
Team training	A structured learning plan covering Selenium WebDriver, Appium, TestNG/JUnit, C#, and Alt Unity Tester, alongside ISTQB exam preparation and hands-on proof-of-concept automation work	Page 19
Release management	Builds promoted through dev, beta and release-candidate stages before a go/no-go call against written exit criteria; a further verification pass on the live build after go-live	Pages 5, 6, 8
Budgeting	Hours and days tracked per month against the plan (576 hours across 43 days on Project A); a staffing change is made as an explicit, written trade-off against schedule and cost, not a silent cut	Page 8
Environment setup	Hardware matrix from low to high-end machines, Jira project and permissions configured before testing starts, network simulators for connectivity conditions — all agreed before day one of execution	Pages 6, 13
SOP creation	Bug Reporting SOP, Testing Procedure and the bug lifecycle itself written down, so execution is repeatable and does not depend on one person's memory	Pages 12, 19

This page introduces no new claims. It presents the project management discipline behind work documented in full detail on the pages referenced.

What I bring to a new studio

First 90 days

Window	Focus	Outcome
Days 1-30: understand	Read the game design document, map the build pipeline, review existing defect data and Jira set-up, play the game in every mode	A baseline: closure rate, reopen rate, defect ageing, escape history
Days 31-60: standardise	Bug report standard, severity and priority as separate mandatory fields, one lifecycle, hardware and resolution matrix, entry and exit criteria	One trusted defect dataset and a signed-off test plan
Days 61-90: scale	Reusable regression suites for progression, save/load and AI; a daily and weekly reporting rhythm; first automation candidates	Faster build verification and a repeatable go/no-go report

Metrics I would put on the dashboard

Metric	Why	My baseline
Defect closure rate and ageing	Shows whether the backlog is shrinking	Project B: 59.2% closed, median unresolved age 35 days
Reopen rate	Fix quality	Project B: 3.9%
Escaped defects (found after handover)	Real measure of test effectiveness	Project A: two escapes (July 2024), both analysed
Test-case pass rate per build	Build health	Project A checkpoint: 94.4%
Ownership spread	Delivery risk	Project B: 84% of defects with one developer

Source index

Artifact	Page	Available as
Test Plan, Testing Procedure, Test Scenarios, Test Cases (204), RTM	6, 15-18, 22	Redacted originals
Daily Report workbook (19 sheets), Game Checklist, Observation log	8-10	Redacted original
Bug triage and test summary (Project B), Jira defect export (152)	11, 22	Anonymised original
Incident report, Postmortem, Testing closure and handover (Project A)	9, 22	Redacted originals
Bug Reporting SOP, Jira reporting structure, Jira admin screenshots	12-14, 22	Redacted originals
Mobile checklist (Android/iOS)	22	Redacted original
Edge-case workbooks (common + genre-specific)	17	Redacted originals
Team Pivot yearly goals workbook	20	Original screenshot and workbook
QA roadmap, weekly report, research notes, tool assessment	6-7, 19, 21	Redacted originals

How these numbers were checked. Project A: the 466 status lines in the daily bug log were de-duplicated to 212 unique tickets. Project B: status, priority, component, reporter, assignee and ageing figures were recomputed from the Jira export and matched the triage report. RTM: all 25 mappings re-read against the test-case workbook. Where sources disagreed, this portfolio uses the figure that can be reproduced.

Next step. Full CV, references and redacted originals are available on request; I am glad to walk through any page of this portfolio, including the parts that did not go to plan. Reach me at mrrahulsharma@yahoo.com or +91-7217704787.